Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу

«Операционные системы»

Группа: М8О-209БВ-25

Студент: Карташев М.М.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 24.09.26

Москва, 2026

Постановка задачи

Вариант 13.

Составить и отладить программу на языке Си, осуществляющую работу с процессами и взаимодействие между ними в операционной системе семейства UNIX (Linux/macOS). Необходимо обрабатывать системные ошибки, которые могут возникнуть в результате работы.

Родительский процесс создаёт два дочерних процесса. Родительский и дочерние процессы должны быть представлены разными программами. Родительский процесс принимает от пользователя строки произвольной длины и пересылает их в pipe1. Процессы child1 и child2 производят работу над строками. Child1 и child2 соединены между собой дополнительным каналом. Child2 пересылает результат своей работы родительскому процессу. Родительский процесс полученный результат выводит в стандартный поток вывода.

Вариант 13: child1 переводит строки в нижний регистр, child2 превращает все пробельные символы в символ «_».

Общий метод и алгоритм решения

Использованные системные вызовы:

- **pid_t fork(void);** – создаёт дочерний процесс, копию родительского.
- **int execv(const char *path, char *const argv[]);** – заменяет образ памяти процесса образом другой исполняемой программы (child1 или child2).
- **int pipe(int fd[2]);** – создаёт неименованный канал: fd[0] для чтения, fd[1] для записи.
- **int dup2(int oldfd, int newfd);** – переназначает файловый дескриптор; используется, чтобы подменить stdin и stdout дочерних процессов концами каналов.
- **int close(int fd);** – закрывает файловый дескриптор.
- **ssize_t read(int fd, void *buf, size_t count);** и **ssize_t write(int fd, const void *buf, size_t count);** – чтение и запись данных в каналы и стандартные потоки.
- **int poll(struct pollfd *fds, nfds_t nfds, int timeout);** – ожидание готовности нескольких дескрипторов (stdin, pipe1, pipe2) одновременно.
- **int fcntl(int fd, int cmd, ...);** – перевод конца pipe1 в неблокирующий режим (O_NONBLOCK).
- **int sigaction(int sig, const struct sigaction *act, struct sigaction *old);** – игнорирование сигнала SIGPIPE, чтобы ошибка записи в закрытый канал обрабатывалась как EPIPE.
- **ssize_t readlink(const char *path, char *buf, size_t size);** – получение полного пути к программе через /proc/self/exe (Linux), чтобы найти рядом лежащие child1 и child2.
- **pid_t waitpid(pid_t pid, int *status, int options);** – ожидание завершения дочернего процесса и получение кода его завершения.
- **void exit(int status);** – завершение процесса с кодом возврата.

Программа состоит из трёх отдельных исполняемых файлов: parent, child1 и child2. Стандартная библиотека C (stdio.h и др.) не используется: весь ввод-вывод выполняется системными вызовами read() и write(), а сообщения об ошибках формируются вручную.

Схема взаимодействия: пользователь → parent → pipe1 → child1 → pipe_mid → child2 → pipe2 → parent → стандартный вывод.

Алгоритм работы родительского процесса:

- Определяет каталог, в котором находится сама программа (через /proc/self/exe, а если его нет, как в macOS, через argv[0]). В этом же каталоге должны находиться child1 и child2.
- Создаёт три канала: pipe1 (parent → child1), pipe_mid (child1 → child2) и pipe2 (child2 → parent).
- Дважды вызывает fork(). В каждом дочернем процессе с помощью dup2() stdin и stdout перенаправляются на нужные концы каналов, все лишние дескрипторы закрываются, после чего вызывается execv() программы child1 или child2.
- Закрывает у себя неиспользуемые концы каналов. Это важно: пока хотя бы один процесс держит открытым конец канала для записи, читающая сторона не получит конец файла (EOF).
- В цикле с помощью poll() ожидает одно из событий: данные от пользователя в stdin, готовность pipe1 к записи или данные в pipe2. Строки пользователя пересылаются в pipe1, а всё, что пришло из pipe2, выводится в stdout.
- Когда пользователь завершает ввод (Ctrl+D), родитель закрывает pipe1. Child1 получает EOF и завершается, закрывая pipe_mid; затем завершается child2 и закрывает pipe2. Родитель читает остаток данных, дожидается обоих процессов через waitpid() и сообщает об их аварийном завершении, если оно произошло.

Child1 читает данные из stdin порциями, переводит латинские буквы A–Z и русские буквы А–Я, Ё (в кодировке UTF-8) в нижний регистр и пишет результат в stdout. Так как русская буква в UTF-8 занимает два байта, а порция может оборваться посередине символа, первый байт сохраняется и обрабатывается вместе со следующей порцией.

Child2 читает данные из stdin и заменяет пробельные символы (пробел, \t, \v, \f, \r) на «_». Символ перевода строки \n не заменяется, так как это разделитель строк, а не часть строки.

Строки могут быть произвольной длины, поэтому данные передаются потоком, а не целыми строками. Чтобы при очень длинной строке не возникла взаимная блокировка (родитель ждёт освобождения pipe1, а child2 ждёт, пока родитель прочитает pipe2), запись в pipe1 сделана неблокирующей, а родитель одновременно читает pipe2 с помощью poll().

Все системные вызовы проверяются на ошибки. При ошибке в stderr выводится сообщение с кодом errno, а процесс завершается с кодом 1.

Код программы

common.h

```
#ifndef LAB1_COMMON_H
#define LAB1_COMMON_H

#include <errno.h>
#include <stdlib.h>
#include <unistd.h>

#define BUF_SIZE 4096

static inline void str_append(char *dst, size_t size, size_t *pos, const char
*src) {
    while (*src != '\0' && *pos + 1 < size)
        dst[(*pos)++] = *src++;
    dst[*pos] = '\0';
}
```

```
static inline void int_append(char *dst, size_t size, size_t *pos, int value) {
    char digits[16];
    int count = 0;
    unsigned int v = value < 0 ? 0u - (unsigned int)value : (unsigned
int)value;

    do {
        digits[count++] = (char)('0' + v % 10);
        v /= 10;
    } while (v != 0);

    if (value < 0)
        digits[count++] = '-';

    char out[17];
    for (int i = 0; i < count; ++i)
        out[i] = digits[count - 1 - i];
    out[count] = '\0';

    str_append(dst, size, pos, out);
}

static inline int write_all(int fd, const char *buf, size_t size) {
    size_t done = 0;
    while (done < size) {
        ssize_t n = write(fd, buf + done, size - done);
        if (n == -1) {
            if (errno == EINTR)
                continue;
            return -1;
        }
        done += (size_t)n;
    }
    return 0;
}

static inline void print_error(const char *prog, const char *msg) {
    int saved_errno = errno;
    char out[512];
    size_t len = 0;

    str_append(out, sizeof(out), &len, prog);
    str_append(out, sizeof(out), &len, ": error: ");
    str_append(out, sizeof(out), &len, msg);
    str_append(out, sizeof(out), &len, " (errno=");
    int_append(out, sizeof(out), &len, saved_errno);
    str_append(out, sizeof(out), &len, ")\n");

    write_all(STDERR_FILENO, out, len);
}

static inline void die(const char *prog, const char *msg) {
    print_error(prog, msg);
    exit(EXIT_FAILURE);
}

#endif
```

parent.c

```
#include <fcntl.h>
#include <poll.h>
#include <signal.h>
#include <sys/wait.h>
```

```
#include "common.h"

static const char PROG[] = "parent";
static const char CHILD1_NAME[] = "child1";
static const char CHILD2_NAME[] = "child2";

static void get_program_dir(const char *argv0, char *dir, size_t size) {
    ssize_t len = readlink("/proc/self/exe", dir, size - 1);

    if (len == -1) {
        len = 0;
        while (argv0[len] != '\0' && (size_t)len < size - 1) {
            dir[len] = argv0[len];
            ++len;
        }
    }
    dir[len] = '\0';

    while (len > 0 && dir[len] != '/')
        --len;

    if (dir[len] == '/') {
        dir[len] = '\0';
    } else {
        dir[0] = '.';
        dir[1] = '\0';
    }
}

static pid_t spawn(const char *dir, const char *name, int in_fd, int out_fd,
                   const int *fds, size_t fds_count) {
    pid_t pid = fork();

    if (pid == -1)
        die(PROG, "failed to fork");

    if (pid == 0) {
        if (dup2(in_fd, STDIN_FILENO) == -1)
            die(name, "failed to redirect stdin");
        if (dup2(out_fd, STDOUT_FILENO) == -1)
            die(name, "failed to redirect stdout");

        for (size_t i = 0; i < fds_count; ++i)
            close(fds[i]);

        char path[1024];
        size_t len = 0;
        str_append(path, sizeof(path), &len, dir);
        str_append(path, sizeof(path), &len, "/");
        str_append(path, sizeof(path), &len, name);

        char *const args[] = {(char *)name, NULL};
        execv(path, args);

        die(name, "failed to exec child program");
    }

    return pid;
}

static int wait_child(pid_t pid, const char *name) {
    int status;

    while (waitpid(pid, &status, 0) == -1) {
        if (errno != EINTR) {
```

```
            print_error(PROG, "failed to wait for child");
            return -1;
        }
    }

    if (WIFEXITED(status) && WEXITSTATUS(status) == EXIT_SUCCESS)
        return 0;

    char msg[128];
    size_t len = 0;
    str_append(msg, sizeof(msg), &len, PROG);
    str_append(msg, sizeof(msg), &len, ": ");
    str_append(msg, sizeof(msg), &len, name);
    if (WIFEXITED(status)) {
        str_append(msg, sizeof(msg), &len, " exited with code ");
        int_append(msg, sizeof(msg), &len, WEXITSTATUS(status));
    } else {
        str_append(msg, sizeof(msg), &len, " was killed by signal ");
        int_append(msg, sizeof(msg), &len, WTERMSIG(status));
    }
    str_append(msg, sizeof(msg), &len, "\n");
    write_all(STDERR_FILENO, msg, len);
    return -1;
}

int main(int argc, char **argv) {
    (void)argc;

    struct sigaction sa = {0};
    sa.sa_handler = SIG_IGN;
    if (sigaction(SIGPIPE, &sa, NULL) == -1)
        die(PROG, "failed to ignore SIGPIPE");

    char dir[1024];
    get_program_dir(argv[0], dir, sizeof(dir));

    int pipe1[2];
    int pipe_mid[2];
    int pipe2[2];
    if (pipe(pipe1) == -1 || pipe(pipe_mid) == -1 || pipe(pipe2) == -1)
        die(PROG, "failed to create pipe");

    const int all_fds[] = {pipe1[0], pipe1[1], pipe_mid[0], pipe_mid[1],
pipe2[0], pipe2[1]};
    const size_t all_count = sizeof(all_fds) / sizeof(all_fds[0]);

    pid_t child1 = spawn(dir, CHILD1_NAME, pipe1[0], pipe_mid[1], all_fds,
all_count);
    pid_t child2 = spawn(dir, CHILD2_NAME, pipe_mid[0], pipe2[1], all_fds,
all_count);

    close(pipe1[0]);
    close(pipe_mid[0]);
    close(pipe_mid[1]);
    close(pipe2[1]);

    int to_child = pipe1[1];
    int from_child = pipe2[0];

    int flags = fcntl(to_child, F_GETFL);
    if (flags == -1 || fcntl(to_child, F_SETFL, flags | O_NONBLOCK) == -1)
        die(PROG, "failed to make pipe non-blocking");

    char in_buf[BUF_SIZE];
    size_t in_len = 0;
    size_t in_pos = 0;
```

```
    char out_buf[BUF_SIZE];
    int stdin_open = 1;
    int status = EXIT_SUCCESS;

    while (from_child != -1) {
        struct pollfd fds[2];
        nfds_t nfds = 0;

        fds[nfds].fd = from_child;
        fds[nfds].events = POLLIN;
        ++nfds;

        if (in_pos < in_len) {
            fds[nfds].fd = to_child;
            fds[nfds].events = POLLOUT;
            ++nfds;
        } else if (stdin_open) {
            fds[nfds].fd = STDIN_FILENO;
            fds[nfds].events = POLLIN;
            ++nfds;
        }

        if (poll(fds, nfds, -1) == -1) {
            if (errno == EINTR)
                continue;
            die(PROG, "failed to poll");
        }

        if (fds[0].revents & (POLLIN | POLLHUP | POLLERR)) {
            ssize_t n = read(from_child, out_buf, sizeof(out_buf));
            if (n == -1 && errno != EINTR) {
                die(PROG, "failed to read from pipe2");
            } else if (n == 0) {
                close(from_child);
                from_child = -1;
            } else if (n > 0 && write_all(STDOUT_FILENO, out_buf, (size_t)n) ==
-1) {
                die(PROG, "failed to write to stdout");
            }
        }

        if (nfds < 2 || fds[1].revents == 0)
            continue;

        if (fds[1].fd == STDIN_FILENO) {
            ssize_t n = read(STDIN_FILENO, in_buf, sizeof(in_buf));
            if (n == -1 && errno != EINTR)
                die(PROG, "failed to read from stdin");

            if (n == 0) {
                stdin_open = 0;
                close(to_child);
                to_child = -1;
            } else if (n > 0) {
                in_len = (size_t)n;
                in_pos = 0;
            }
        } else {
            ssize_t n = write(to_child, in_buf + in_pos, in_len - in_pos);
            if (n == -1) {
                if (errno == EAGAIN || errno == EINTR)
                    continue;
                print_error(PROG, "failed to write to pipe1");
                status = EXIT_FAILURE;
                in_pos = in_len = 0;
                stdin_open = 0;
```

```
                close(to_child);
                to_child = -1;
            } else {
                in_pos += (size_t)n;
            }
        }
    }

    if (to_child != -1)
        close(to_child);

    if (wait_child(child1, CHILD1_NAME) == -1)
        status = EXIT_FAILURE;
    if (wait_child(child2, CHILD2_NAME) == -1)
        status = EXIT_FAILURE;

    return status;
}
```

child1.c

```
#include "common.h"

static const char PROG[] = "child1";

static size_t to_lower(const unsigned char *in, size_t size, unsigned char
*out, int *carry) {
    size_t o = 0;
    size_t i = 0;

    while (i < size) {
        unsigned char c = in[i];

        if (*carry != -1) {
            unsigned char lead = (unsigned char)*carry;
            *carry = -1;
            if (lead == 0xD0 && c >= 0x90 && c <= 0x9F) {
                out[o++] = 0xD0;
                out[o++] = c + 0x20;
            } else if (lead == 0xD0 && c >= 0xA0 && c <= 0xAF) {
                out[o++] = 0xD1;
                out[o++] = c - 0x20;
            } else if (lead == 0xD0 && c == 0x81) {
                out[o++] = 0xD1;
                out[o++] = 0x91;
            } else {
                out[o++] = lead;
                out[o++] = c;
            }
            ++i;
            continue;
        }

        if (c >= 'A' && c <= 'Z') {
            out[o++] = c + ('a' - 'A');
        } else if (c == 0xD0) {
            *carry = c;
        } else {
            out[o++] = c;
        }
        ++i;
    }

    return o;
```

```
}

int main(void) {
    unsigned char in[BUF_SIZE];
    unsigned char out[BUF_SIZE + 1];
    int carry = -1;
    ssize_t bytes;

    while ((bytes = read(STDIN_FILENO, in, sizeof(in))) != 0) {
        if (bytes == -1) {
            if (errno == EINTR)
                continue;
            die(PROG, "failed to read from stdin");
        }

        size_t n = to_lower(in, (size_t)bytes, out, &carry);
        if (write_all(STDOUT_FILENO, (const char *)out, n) == -1)
            die(PROG, "failed to write to stdout");
    }

    if (carry != -1) {
        unsigned char last = (unsigned char)carry;
        if (write_all(STDOUT_FILENO, (const char *)&last, 1) == -1)
            die(PROG, "failed to write to stdout");
    }

    return EXIT_SUCCESS;
}
```

child2.c

```
#include "common.h"

static const char PROG[] = "child2";

static int is_space(char c) {
    return c == ' ' || c == '\t' || c == '\v' || c == '\f' || c == '\r';
}

int main(void) {
    char buf[BUF_SIZE];
    ssize_t bytes;

    while ((bytes = read(STDIN_FILENO, buf, sizeof(buf))) != 0) {
        if (bytes == -1) {
            if (errno == EINTR)
                continue;
            die(PROG, "failed to read from stdin");
        }

        for (ssize_t i = 0; i < bytes; ++i) {
            if (is_space(buf[i]))
                buf[i] = '_';
        }

        if (write_all(STDOUT_FILENO, buf, (size_t)bytes) == -1)
            die(PROG, "failed to write to stdout");
    }

    return EXIT_SUCCESS;
}
```

Makefile

```
CC      ?= cc
CFLAGS  ?= -std=c11 -Wall -Wextra -Wpedantic -O2
CFLAGS  += -D_POSIX_C_SOURCE=200809L -D_DEFAULT_SOURCE

BUILD   := build
SRC     := src

all: $(BUILD)/parent $(BUILD)/child1 $(BUILD)/child2

$(BUILD)/%: $(SRC)/%.c $(SRC)/common.h | $(BUILD)
    $(CC) $(CFLAGS) -o $@ $<

$(BUILD):
    mkdir -p $@

run: all
    ./$(BUILD)/parent

test: all
    ./tests/run_tests.sh

clean:
    rm -rf $(BUILD)

.PHONY: all run test clean
```

Протокол работы программы

Сборка:

```
$ make
mkdir -p build
cc -std=c11 -Wall -Wextra -Wpedantic -O2 -D_POSIX_C_SOURCE=200809L
-D_DEFAULT_SOURCE -o build/parent src/parent.c
cc -std=c11 -Wall -Wextra -Wpedantic -O2 -D_POSIX_C_SOURCE=200809L
-D_DEFAULT_SOURCE -o build/child1 src/child1.c
cc -std=c11 -Wall -Wextra -Wpedantic -O2 -D_POSIX_C_SOURCE=200809L
-D_DEFAULT_SOURCE -o build/child2 src/child2.c
```

Тестирование:

```
$ ./build/parent
Hello World
hello_world
ОПЕРАЦИОННЫЕ   Системы
операционные___системы
MAI\tLab  №1 Variant 13
mai_lab__№1_variant_13


   LEADING and trailing
___leading_and_trailing___
^D
$ echo $?
0
```

Обработка ошибки: программа child2 отсутствует:

```
$ rm build/child2
$ echo "test" | ./build/parent
child2: error: failed to exec child program (errno=2)
child1: error: failed to write to stdout (errno=32)
```

```
parent: child1 exited with code 1
parent: child2 exited with code 1
$ echo $?
1
```

Автотесты:

```
$ make test
PASS: простая строка
PASS: несколько строк
PASS: табуляция и CR
PASS: пустая строка
PASS: без пробелов
PASS: только пробелы
PASS: кириллица
PASS: без перевода строки
PASS: пустой ввод
PASS: длинная строка (2 МБ)
PASS: запуск из другого каталога
```

Вывод

В ходе лабораторной работы были получены практические навыки управления процессами в ОС: создание процессов с помощью fork(), запуск в них отдельных программ через execv(), организация обмена данными между процессами через неименованные каналы и перенаправление стандартных потоков с помощью dup2(). Была реализована цепочка из трёх процессов, в которой строки последовательно обрабатываются двумя дочерними процессами. Основные трудности: необходимость закрывать все неиспользуемые концы каналов, иначе процессы не получают EOF и зависают; возможная взаимная блокировка при передаче длинных строк, которая была решена через poll() и неблокирующую запись; а также запрет на stdio.h, из-за которого вывод сообщений и сборку строк пришлось реализовать вручную через write().